

STRING SLICING, INDEXING & RANGING ①

1. What is the output of the following, and why?

`S = "Python"`

`Print(s[0], s[-1], s[6])`

O/P

`s[0]` $\rightarrow$ P

`s[-1]` $\rightarrow$ n

`s[6]` $\rightarrow$ Error because it is out of range in the given string

2. What is b/w positive & negative indexing?

`S = "Hello"`

What does `s[-2]` return?

O/p

`s[-2]` = 'l'

Positive indexing $\rightarrow$ It is to count the elements from left to right in a list or string

Negative indexing $\rightarrow$ It is to access the elements from right to left starting @ -1 index [From last element]

3. What happens when you access an index that is out of range in a string? How is this different from slicing out of range?

`S = "code"`

`print(s[10])` # Error as the index is out of range

`print(s[1:10])` # 'ode'

↳ slice prints available elements.

4. How do you find the index of the first and last occurrence of a character in a string using indexing concepts?

O/p

`S = "mississippi"`
0 1 2 3 4 5 6 7 8 9 10

First index of S = 2
last index of S = 6

(2)

5. What is the output of the following and explain how negative indexing resolves?

S = "a⁻⁶b⁻⁵c⁻⁴d⁻³e⁻²f⁻¹"
 0 1 2 3 4 5

Print (s[-1], s[-3], s[-6])

O/P

 ↓ ↓ ↓
 f d a

As it is a negative indexing, it always starts from right to left

SLICING BASICS:

6. What is the syntax for string slicing in python?
Explain each part of s[start:stop:step]

O/P

* It is to get specific position of given string

Start → index where the slicing starts [included]

stop → index where " " stops

Step → ~~index~~

It is to move by step size
(or)

How many index/letters to skip

7. What is the output of the following slices, and why?

$S = \overset{0\ 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9}{\text{"Helloworld"}}$

`print(s[0:5])` $\rightarrow$ 'Hello'

`print(s[5:])` $\rightarrow$ 'world'

`print(s[:5])` $\rightarrow$ 'Hello'

`print(s[:])` $\rightarrow$ 'Helloworld'

8. What happens when start is greater than stop in a slice with a positive step? What is output?

`S = "python"`

`print(s[4:1])`

In Python, slicing moves left to right only when step is positive

$4 \rightarrow 1$ (backward direction)

So it cannot move and returns empty space.

9. How do you extract the last 3 characters of a string using slicing without knowing its length?

O/p

yes it is possible to extract the last

3 characters using negative index

10. What is the difference between these two slices? What does each return?

$S = \overset{-8}{a}\overset{-7}{b}\overset{-6}{c}\overset{-5}{d}\overset{-4}{e}\overset{-3}{f}\overset{-2}{g}\overset{-1}{h}$
 $\underset{0}{a}\underset{1}{b}\underset{2}{c}\underset{3}{d}\underset{4}{e}\underset{5}{f}\underset{6}{g}\underset{7}{h}$

Print($S[::1]$) $\rightarrow$ abcdefgh

Print($S[::2]$) $\rightarrow$ aceg

Print($S[1::2]$) $\rightarrow$ bdfh

STEP 8 REVERSE SLICING:

11. How do you reverse a string using slicing? What is the step value that achieves this?

$S = \text{"python"}$

Reverse it using slicing

O/P

Print($S[::-1]$)

$\rightarrow$ nohtyp $\rightarrow$ step value is (-1)

12. What is the output of the following, and explain how negative step works?

$S = \overset{-8}{a}\overset{-7}{b}\overset{-6}{c}\overset{-5}{d}\overset{-4}{e}\overset{-3}{f}\overset{-2}{g}\overset{-1}{h}$
 $\underset{0}{a}\underset{1}{b}\underset{2}{c}\underset{3}{d}\underset{4}{e}\underset{5}{f}\underset{6}{g}\underset{7}{h}$

Print($S[::-1]$) $\rightarrow$ hgfedcba

Print($S[::-2]$) $\rightarrow$ hfdb

Print($S[6:1:-1]$) $\rightarrow$ gfedc

The element from the given string is from right to left (backward)

13. How do you extract every 3rd character from a string starting from index 1?

$S = "a1b2c3d4eS"$

D/P #1 Expected output "12345"

Print (s[^{1::2}~~0:5~~]) $\Rightarrow$ 123.45

14. What is the output of the following negative step slices?

`s = 'python'`

`Print(s[s::-1])` → nohty

`Print(s[5::-1])` → nohtyp

`print(s[:0:-1])` → nohty.

15. How do slices handle start & stop values that exceed string boundaries with a negative step?

s = "hello"

`print(s[100::-1])` → ollerh

`print(s[::-100])` → 0

INTERMEDIATE SLICING CHALLENGES:

16. How do you extract the middle portion of a string dynamically, regardless of its length?

```
print(s[2:-2])
```

cdefgh

`s = "abcdefghi"`
 0 1 2 3 4 5 6 7 8 9
 # extract characters from index 2 to second last. (exclusive)

17. How do you use slicing to check if a string is a palindrome in a single line?

```
s = "racecar"
# check palindrome using slicing
```

O/P `print(s[::-1])` → True
 'racecar' → True

18. What is the O/P of the following, and why does Python not raise an error?

```
s = 'Hello'
print(s[1:100]) → ello
print(s[-100:3]) → Hel
print(s[100:200]) → empty space
```

slicing won't give error because it handles out of range in the string

19. How do you replace a portion of a string using slicing (since strings are immutable)?

```
S = "Hello world"
    0 1 2 3 4 5 6 7 8 9 10
# replace "world" with "python" using slicing
and concatenation
```

O/P print(S[:6] + 'python') $\Rightarrow$ Hello python

Advanced Slicing concepts:

20. How does python's slice() object work, and how can you use it as a reusable slice?

```
S = 'abcdefghij'
    0 1 2 3 4 5 6 7 8 9
my_slice = slice(1, 8, 2)
print(s[my_slice])
```

What is the output, and how can this be reused on other strings?

O/P b d f h